

Profiling & Alleviating Performance Bottlenecks in Zeek

Jyotsna Arunkumar

Arnav Batra

Matthew Sickler

The University of Texas at Austin
Austin, USA

ABSTRACT

(The two-page proposal should describe the problem, its relationship to the course, the proposed work, the responsibilities of each team member, the resources required, and how the outcome will be evaluated. Your proposal should convince us that the project is relevant, technically meaningful, feasible within one semester, and supported by a concrete evaluation plan.

Engineering-track proposals should identify the intended users and their current workflow, survey existing tools, explain the missing capability, define the proposed interface and implementation, and describe how you will demonstrate correctness and usefulness. The proposal must also explain how the project can be developed and evaluated without specialized hardware or a multi-GPU cluster.)

1 RELATED WORK

(include summary of related work and existing gaps in research here)

2 PROBLEM & MOTIVATION

First developed under the name "Bro" in the 1990s, Zeek is an open-source network security monitoring platform that passively analyzes real-time network traffic, performing actions including protocol parsing, connection tracking, event generation, scripting, and logging. According to the Zeek Project website, it has over 10,000 deployments worldwide [1].

However, Zeek's packet-processing cost can vary substantially with traffic rate, packet size, flow churn, protocol mix, enabled analyzers, and scripts. Within a single Zeek worker, these costs can create processing bottlenecks, which may lead to incomplete visibility into network traffic as packets are dropped from processing queues. In deployments with multiple workers, which are typically assigned packets via flow hashing, this variable cost can also lead to load imbalances across workers. As a result, the actual processing capacity of a worker cluster may be lower than the theoretical maximum, as seen in metrics including throughput and packet drops.

Zeek's users range from network security operators of enterprise networks to academic researchers. Our hope is to mitigate the bottlenecks that prevent them from using the tool and its underlying resources to the fullest potential.

By improving on Zeek's packet-processing performance, its users will be able to analyze larger workloads with more accuracy and gain better visibility into network traffic.

3 PROPOSED WORK

("define the proposed interface and implementation")

4 EVALUATION

We will evaluate the project using controlled workloads that vary four traffic characteristics: packet rate, packet size, flow churn, and protocol mix. Profiling these variables may help explain why some flows are more expensive to process than others.

Our evaluation will focus on whether these workload characteristics contribute to bottlenecks and load imbalances, and how our proposed mitigation improves performance.

Our main metrics will be:

- packet-processing throughput,
- packet-drop rate,
- CPU utilization,
- worker load imbalance,
- memory consumption
- any overhead introduced by the proposed mitigation

4.1 75% Goal

We will characterize how Zeek performance and worker imbalance change by running workloads with different packet rates, packet sizes, flow churn, and protocol mixes. Using these experiments, we expect to narrow down to one reproducible performance bottleneck.

We will then implement a simple mitigation that addresses the identified bottleneck. For example, if worker imbalance is the targeted issue, we will implement cost-aware flow-assignment to send new flows to a less-loaded Zeek worker.

We will compare the proposed mitigation against Zeek's existing behavior under the same workload. We will consider this goal successful if the mitigation demonstrates a measurable performance improvement, like reduced worker imbalance, fewer packet drops, lower CPU utilization, or higher throughput.

4.2 100% Goal

We will design and implement an improved mitigation based on the bottleneck identified during profiling and on the results of the 75% implementation. For worker imbalance, we will iterate on our cost-aware flow-assignment strategy to consider the estimated processing cost of a new flow and the current load of each Zeek worker while preserving flow affinity. For other bottlenecks, we will design an improved mitigation.

We will evaluate the improved mitigation across different types of network traffic, comparing it against Zeek’s existing performance. We will measure metrics such as throughput, packet drops, CPU utilization, worker-load imbalance, memory consumption, and overhead. Using these metrics, we will characterize the observed benefits and possible limitations of the mitigation.

4.3 125% Goal

We will extend the 100% solution with a more dynamic implementation that responds to changes in workload behavior. One possible extension is to predict a new flow’s processing cost using information from its first few packets (e.g. packet size, header metadata, transport protocol). We will evaluate whether this prediction improves worker assignment compared with the 100% solution.

For other bottlenecks, the 125% goal will extend the existing mitigation, possibly by identifying unusually expensive flows or dynamically adding/removing workers.

5 REQUIRED RESOURCES

The proposed work does not require specialized hardware or a multi-GPU cluster.

Our main resources will include:

- **CloudLab** for compute resources
- **tcpreplay** for existing packet captures
- **iperf3** for generating traffic with variable packet sizes and flow patterns with ease of use
- **DPDK** for generating high-speed traffic and greater control, as needed
- **Linux perf** for profiling CPU usage
- **zeekctl top** for measuring per-worker CPU and memory utilization
- **Zeek’s Prometheus tool** for logging per-worker CPU metrics during automated experiments

6 TEAM MEMBER RESPONSIBILITIES

Our goal is to create as meaningful a solution as possible within the limited timeframe. To achieve this, we need to operate efficiently as a team, and the basic responsibilities of each team member are structured accordingly. We will prioritize strong communication and aim to make significant

progress early in the semester. We plan to meet in-person at least once a week. During these meetings, we will craft goals for the week and develop designs based on input from all three team members. We will then complete our work separately before regrouping. Finally, we plan to check in periodically with Dr. Sengupta to get feedback on our direction and identify any aspects we may be overlooking.

The following is an example of how we intend to divide the proposed work among the three teammates. One person will likely take the lead in setting up the Zeek and packet-generating nodes in CloudLab, but all three of us will participate so that each of us can easily modify the profiling framework as needed. Next, we will each pick one or two of the packet characteristics to profile, collecting the data required for the remaining parts of the project. Together, we will then identify the bottleneck that appears most promising to address. For each mitigation type explained in the “Evaluation” section (basic, improved, and dynamic), we will independently develop potential mitigation strategies, combine our ideas into a single solution focusing, and implement it together. For the report and presentation, all three of us will contribute to an initial draft. Then, one or two team members will focus on refining the report while the others develop the presentation.

REFERENCES

- [1] Zeek Project. [n. d.]. *Zeek*. <https://zeek.org/>